Universidade do Minho
Escola de Engenharia

Licenciatura em Engenharia Informática

Relatório 2º Fase

Laboratórios de Informática III

Ano Letivo de 2025/2026

a104539, João Pedro Ferreira e Ferreira
a103997, Maria Leonor Carvalho da Cunha
a104530, Tiago Rodrigues Barros

Braga, janeiro de 2026

Índice

Índice i

1 Introdução 1

2 Desenvolvimento 2
2.1 Ajustes Realizados 2
2.2 Estrutura Aplicada 2
2.3 Funcionamento do Projeto 2
2.4 Programa de Testes 3
2.5 Modularização, Encapsulamento e Estruturas de Dados 3
2.6 Queries 4
2.6.1 Query 1 - Resumo de um Aeroporto 4
2.6.2 Query 2 – Top N aeronaves com mais voos realizados 4
2.6.3 Query 3 – Aeroporto com mais partidas num intervalo de datas 5
2.6.4 Query 4 - Passageiro que esteve mais tempo no top 10 de passageiros que mais gastaram em viagens durante um período 6
2.6.5 Query 5 - Top N companhias aéreas com mais tempo de atraso médio por voo . . 7
2.6.6 Query 6 - Listar o aeroporto de destino mais comum para passageiros de uma determinada nacionalidade 7
2.7 Otimizações e Limitações 8
2.8 Dificuldades Sentidas 8

3 Conclusão 8

1 Introdução

O presente documento descreve o trabalho desenvolvido na 2º Fase do Projeto Final da Unidade Curricular Laboratórios de Informática III, pertencente ao 2º Ano da Licenciatura em Engenharia Informática da Universidade do Minho, no ano letivo 2025/2026.

Após a conclusão da 1ª fase, na qual foram implementados o parsing e validação dos dados, o modo batch e um conjunto inicial de queries, esta segunda fase teve como principal objetivo a conclusão das restantes interrogações propostas, bem como a melhoria da arquitetura da aplicação e a adaptação ao dataset fornecido pela equipa docente.

O projeto tem como objetivo principal a implementação de uma base de dados em memória, a partir de ficheiros .csv fornecidos pela equipa docente. Estes ficheiros contém informações relativas a viagens (voos, reservas, passageiros, etc.), e o propósito central é armazenar e processar essses dados, permitindo pesquisas e associações entre as várias entidades.

Durante a realização do trabalho foram aplicados conceitos fundamentais como modularidade, encapsulamento, reutilização e abstração de dados.

2 Desenvolvimento

Nesta fase foi possível consolidar a arquitetura definida anteriormente, completar todas as queries propostas e introduzir otimizações que permitiram melhorar o desempenho da aplicação, especialmente no processamento de grandes volumes de dados.

Para além do modo batch, foi garantido o correto funcionamento global da aplicação, mantendo elevados padrões de organização do código, documentação e gestão de memória.

2.1 Ajustes Realizados

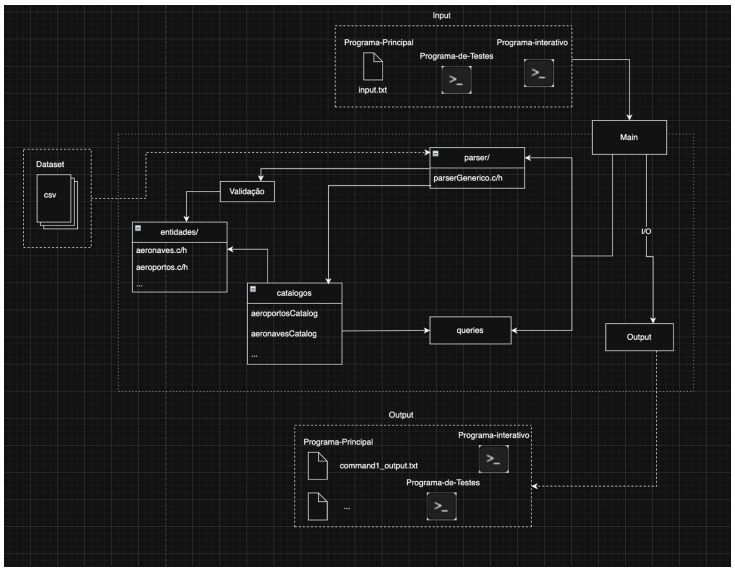
Ao longo da 2ª fase foram efetuados diversos ajustes, tanto com base no feedback obtido na defesa intermédia como devido à necessidade de adaptação ao large dataset.

Entre as principais alterações realizadas destacam-se:

- Reformulação e otimização de algumas estruturas de dados para reduzir o consumo de memória;
- Alteração de tipos de dados para equivalentes mais leves sempre que possível;
- Introdução de estratégias de ordenação mais eficientes em queries com grande volume de dados.

2.2 Estrutura Aplicada

O desenvolvimento do projeto seguiu a estrutura modular proposta pelos docentes, permitindo uma separação clara entre os diferentes componentes do sistema e garantindo maior organização e manutenção do código. Os principais módulos incluem catálogos de voos, aeroportos, passageiros, reservas e aeronaves.



2.3 Funcionamento do Projeto

Na primeira fase, aplicação identificava o modo de execução através do número de argumentos recebidos e tinha sido apenas implementado o modo batch.

Logo no início, são criadas, dentro do módulo database, todas as estruturas necessárias ao armazenamento dos dados. A base de dados em memória é então preenchida com a informação proveniente dos ficheiros .csv, cujo diretório é indicado no primeiro argumento do programa.

A leitura é efetuada por um parser genérico, que recebe apontadores para funções responsáveis pela validação, tratamento e inserção dos dados nos catálogos. Se algum registo for inválido, o parser aciona uma função de gestão de erros também passada como parâmetro.

Em seguida, é feito o parsing dos comandos — obtendo o número da query, o formato de saída e os respetivos argumentos. As queries utilizam funções de acesso e listagem dos catálogos.

A escrita dos resultados é tratada por um módulo de output genérico, capaz de exportar em formato CSV ou tabular (com a flag F). Este módulo aceita funções variadas que definem a formatação e os campos de saída, gravando as respostas num buffer temporário, que é posteriormente escrito em disco.

Após a execução completa, o programa realiza uma limpeza total da memória, libertando todas as estruturas alocadas e garantindo a ausência de memory leaks.

Para além do modo batch, foi também implementado, agora na segunda fase, um modo de execução interativo, conforme especificado no enunciado. Neste modo, o utilizador é guiado através de menus no terminal, podendo carregar dinamicamente diferentes conjuntos de dados e executar múltiplas queries de forma iterativa.

Após o carregamento do dataset, é apresentado um menu com todas as queries disponíveis, permitindo ao utilizador selecionar a operação pretendida. Para cada query, o programa solicita apenas os argumentos relevantes, aplicando validações sobre os valores introduzidos (como formatos de datas, códigos de aeroportos, números positivos ou strings não vazias), garantindo robustez na interação.

O utilizador pode ainda escolher o formato de saída pretendido, incluindo o formato alternativo definido no enunciado. Após a execução de cada query, o respetivo resultado é apresentado no terminal, sendo possível executar novas queries sem necessidade de recarregar o dataset.

Este modo interativo melhora a usabilidade da aplicação, permitindo uma exploração mais flexível dos dados, mantendo a separação entre a lógica de processamento das queries e a interface de interação com o utilizador.

2.4 Programa de Testes

Com o objetivo de validar e avaliar o funcionamento do sistema desenvolvido, foi implementado um programa de testes autónomo, responsável por executar e verificar os resultados de todas as queries definidas no enunciado.

O programa-testes recebe como argumentos a diretoria do dataset, um ficheiro com os comandos a executar e uma pasta contendo os outputs esperados. Após a execução do programa principal, os resultados gerados são comparados linha a linha com os ficheiros de referência, sendo contabilizado o número de execuções corretas por tipo de query. Em caso de divergência, é indicada a linha onde ocorre a primeira incongruência.

Para além da validação funcional, o programa-testes mede o tempo total de execução do programa, bem como o consumo máximo de memória. Adicionalmente, é calculado o tempo médio de execução por tipo de query, recorrendo a múltiplas execuções de conjuntos de comandos específicos, permitindo uma avaliação mais precisa do desempenho.

O formato do output foi definido de forma livre, incluindo informação relevante para análise da correção, desempenho e eficiência do sistema.

2.5 Modularização, Encapsulamento e Estruturas de Dados

A modularização foi aplicada de forma consistente ao longo do projeto, dividindo o código em vários módulos distintos e independentes, tais como os módulos de voos, aeroportos, passageiros, (entre

outros) *querys* e *validation*. Cada módulo possui uma responsabilidade bem definida, o que facilita tanto a manutenção como a expansão futura do sistema. Esta abordagem também promove a reutilização de código e reduz a duplicação desnecessária.

O encapsulamento foi igualmente assegurado, garantindo que os dados internos de cada estrutura permanecem protegidos e que quaisquer alterações apenas podem ser realizadas através de funções específicas. Desta forma, minimizam-se efeitos colaterais e diminuem-se as probabilidades de erro. Um exemplo disso é o processo de criação de novos voos, realizado através de funções como `criarVoo()`, que garantem a integridade e consistência da informação armazenada.

As escolhas relativas às estruturas de dados tiveram um papel fundamental no desempenho e eficiência do sistema. A utilização de *HashTables* para o armazenamento permitiu realizar operações de procura, inserção e remoção de forma eficiente.

Estas decisões contribuíram para que o sistema lidasse de maneira eficaz com grandes volumes de dados e múltiplas consultas, melhorando significativamente a sua capacidade de resposta.

2.6 Queries

Em todas as queries na fase 2, como foi pedido no enunciado, foram feitas adaptações a maneira como eram recebidos os inputs, assim como formatados os outputs. O formato de output das queries segue uma de duas variantes consoante o número da query é acrescido (ou não) do carater 'S'. Por omissão, os outputs deverão ser separados por ';', enquanto que o formato alternativo deverá utilizar o separador '='.

2.6.1 Query 1 - Resumo de um Aeroporto

A Query 1 tem como objetivo apresentar um resumo completo de um aeroporto, com base no código recebido como argumento. Este comando recebe um único identificador, correspondente ao código único do aeroporto, e deve devolver os campos *code*, *name*, *city*, *country* e *type*. Caso o código não exista no catálogo, deve ser retornada uma linha vazia.

A implementação inicia-se com o acesso ao *AeroportosCatalog*, onde é realizada uma pesquisa direta através da função `buscarAeroporto()`. Como os aeroportos estão armazenados numa tabela de hash, a operação de consulta ocorre de forma eficiente, com complexidade próxima de O(1).

Se o aeroporto for encontrado, os respetivos campos são extraídos e é construída uma linha formatada com `g_strdup_printf()`. O resultado é posteriormente escrito no ficheiro de saída através da função `save_output()`. Foram também incluídos mecanismos de validação para garantir que a ausência de argumentos ou de catálogos inválidos resulta corretamente numa linha vazia.

Na segunda fase do projeto tivemos que efetuar algumas alterações e a Query 1 não foi exceção. O formato de saída foi alterado e agora o número de passageiros que aterraram no aeroporto e número de passageiros que partiram do aeroporto também se encontram no output.

2.6.2 Query 2 – Top N aeronaves com mais voos realizados

A Query 2 devolve as N aeronaves que realizaram mais voos, considerando apenas os voos cujo estado não seja *cancelado*. A query permite ainda um filtro opcional por fabricante, que restringe os resultados às aeronaves do fabricante indicado.

A implementação começa pela interpretação dos argumentos, verificando a presença ou não de um fabricante. De seguida, o programa invoca a função `voosPorAeronave()`, que gera uma tabela de hash onde:

- A chave corresponde ao identificador único da aeronave;

- O valor representa o número total de voos válidos realizados.

Esta tabela é depois convertida num `GArray` de estruturas `aeronaveVoos` através da função `adicionarAoArray()`, permitindo a sua posterior ordenação com `g_array_sort()`. A ordenação é feita:

1. Por número de voos (ordem decrescente);
2. Em caso de empate, por identificador, em ordem lexicográfica crescente.

Após a ordenação, o programa percorre o array e escreve apenas os top N resultados, aplicando o filtro de fabricante, caso esteja presente. Para cada aeronave são obtidos os campos *manufacturer* e *model* através do catálogo de aeronaves, construindo assim cada linha de saída.

No final, todas as estruturas auxiliares são libertadas, assegurando a inexistência de fugas de memória. A combinação entre tabelas de hash e arrays ordenáveis permite uma solução eficiente para volumes de dados elevados.

Relativamente à Query 2, na segunda fase, foi introduzida uma alteração estrutural com impacto direto no desempenho. Na versão inicial, a query percorria todos os voos do catálogo sempre que era executada, utilizando uma HashTable para contabilizar o número de voos por aeronave, convertendo posteriormente essa informação para um array que era ordenado a cada invocação.

Na versão atual, o número de voos por aeronave passou a ser mantido incrementalmente durante a fase de carregamento dos dados. Cada aeronave possui agora um campo dedicado à contagem de voos, atualizado à medida que os voos são inseridos no catálogo. Após o carregamento completo do dataset, esta informação é transferida para um array que é ordenado uma única vez.

Este array ordenado é preservado ao longo da execução do programa, sendo apenas consultado sempre que a Query 2 é chamada. Esta abordagem reduz significativamente o custo computacional da query, melhorando a eficiência global do sistema.

2.6.3 Query 3 – Aeroporto com mais partidas num intervalo de datas

A Query 3 identifica o aeroporto com maior número de partidas num intervalo temporal definido por duas datas. São considerados apenas voos cuja data real de partida (*actual departure*) se encontra dentro do intervalo e cujo estado é diferente de *cancelado*.

A implementação começa pela leitura das datas fornecidas como argumentos. Em seguida, o programa chama a função `partidasPorAeroportoEntreDatas()`, que devolve uma tabela de hash onde:

- A chave é o código do aeroporto;
- O valor é o número total de partidas válidas no intervalo.

Caso não existam partidas registadas para o intervalo indicado, a query retorna uma linha vazia. Se a tabela possuir valores, o programa percorre todas as entradas através de um *hash iterator*, selecionando o aeroporto com:

1. O maior número de partidas;
2. Em caso de empate, o código lexicograficamente menor.

Após determinar o aeroporto vencedor, o código é duplicado (pois a tabela será destruída) e o catálogo é consultado para obter os campos *name*, *city* e *country*. A linha final produzida segue o formato:

```
code,name,city,country,departure_count
```

À semelhança das restantes queries, todas as estruturas temporárias são libertadas, garantindo uma gestão de memória correta em toda a execução.

Na Query 3, a implementação foi ajustada para cumprir o novo enunciado, passando a considerar explicitamente um intervalo de datas definido pelo utilizador, incluindo ambas as datas limite. A contagem de partidas é realizada apenas para voos cujo instante de partida se encontra dentro desse intervalo.

A determinação do aeroporto com maior número de partidas é feita através da iteração sobre o catálogo de aeroportos, recorrendo a uma função de callback que calcula o total de partidas para cada aeroporto. Em situações de empate, foi introduzido um critério de desempate baseado na ordem lexicográfica do código IATA, conforme especificado.

O formato de saída foi igualmente adaptado, mantendo a consistência com o resto do sistema e suportando o formato alternativo quando solicitado.

De forma semelhante ao que foi realizado na Query 2, a Query 3 foi otimizada através da transferência do esforço computacional para a fase de carregamento dos dados. Na versão inicial, o cálculo do número de partidas por aeroporto era efetuado integralmente sempre que a query era executada.

Na versão atual, a informação necessária para contabilizar partidas por aeroporto passou a ser organizada no catálogo de aeroportos, permitindo que a query apenas percorra os aeroportos e consulte os valores já agregados para o intervalo de datas especificado. O aeroporto com maior número de partidas é então determinado através de uma única iteração, aplicando o critério de desempate definido no enunciado.

Esta abordagem reduz o custo de execução da query, tornando-a mais eficiente e consistente com a estratégia adotada nas restantes queries.

2.6.4 Query 4 - Passageiro que esteve mais tempo no top 10 de passageiros que mais gastaram em viagens durante um período

A Query 4 tem como objetivo identificar o passageiro que integrou o *Top 10* de passageiros com maior gasto em viagens durante o maior número de semanas, considerando opcionalmente um intervalo temporal definido pelo utilizador. O cálculo do *Top 10* é efetuado semanalmente, tendo por base o montante total gasto por cada passageiro em reservas, sendo as semanas delimitadas entre domingo e sábado, de acordo com a data de partida estimada dos voos.

A abordagem adotada inicia-se pela recolha dos gastos semanais de todos os passageiros. Para cada passageiro, são identificadas as semanas em que existiu gasto associado, calculando-se o valor total gasto em cada uma dessas semanas. Quando é fornecido um intervalo de datas, apenas são consideradas as semanas que intersectam total ou parcialmente esse intervalo temporal, garantindo o cumprimento do enunciado.

Os gastos são organizados numa *HashTable* onde cada chave representa o identificador de uma semana e o respetivo valor corresponde a um *array* dinâmico com pares *(passageiro, gasto)*. Para cada semana, este array é ordenado por ordem decrescente de gasto, recorrendo a um critério de desempate baseado no número do documento do passageiro em ordem crescente. Após a ordenação, apenas os dez primeiros passageiros são considerados como pertencentes ao *Top 10* dessa semana.

De seguida, é mantida uma estrutura auxiliar que contabiliza o número de vezes que cada passageiro aparece no *Top 10* semanal. Esta contagem é incrementada sempre que um passageiro surge entre os dez primeiros classificados de uma determinada semana.

Após o processamento de todas as semanas relevantes, é selecionado o passageiro com o maior número de presenças no *Top 10*. Em situações de empate, é escolhido o passageiro com o menor

identificador (número de documento), de acordo com o critério definido no enunciado. Finalmente, são apresentados no output os dados identificativos do passageiro vencedor, juntamente com o número total de semanas em que integrou o *Top 10*.

A complexidade é dominada pela ordenação dos passageiros em cada semana, resultando numa complexidade total de $O(S \cdot P \log P)$, onde S representa o número de semanas distintas e P o número de passageiros.

Infelizmente não conseguimos por a Query 4 a funcionar, deixamos em comentário.

2.6.5 Query 5 - Top N companhias aéreas com mais tempo de atraso médio por voo

A Query 5 tem como objetivo listar as N companhias aéreas com maior atraso médio por voo, considerando apenas voos cujo estado seja *Delayed*. O atraso de cada voo é calculado, em minutos, como a diferença entre a data real de partida *actual departure*) e a data estimada (*departure*).

A implementação começa por validar o valor de *N*. De seguida, o catálogo de voos é percorrido para calcular, para cada companhia aérea, o número total de voos atrasados e o atraso acumulado. A partir destes valores é determinado o atraso médio.

Os resultados são armazenados numa *GPtrArray* de estruturas auxiliares, que é posteriormente ordenada de forma decrescente pelo atraso médio. Em caso de empate, as companhias aéreas são ordenadas por ordem alfabética do seu identificador, conforme especificado no enunciado.

Após a ordenação, são escritos no ficheiro de output apenas os primeiros *N* resultados, sendo o atraso médio apresentado com três casas decimais. Esta abordagem garante eficiência e escalabilidade, mesmo para conjuntos de dados de grande dimensão.

A complexidade total da query é $O(V + A \log A)$, sendo V o número de voos e A o número de companhias aéreas.

2.6.6 Query 6 - Listar o aeroporto de destino mais comum para passageiros de uma determinada nacionalidade

A Query 6 permite identificar o aeroporto de destino mais frequente entre passageiros de uma determinada nacionalidade. O comando recebe como argumento a nacionalidade dos passageiros e devolve o código IATA do aeroporto mais comum, bem como o número de passageiros associados. Apenas são considerados voos cujo estado não seja *Cancelled*.

A implementação inicia-se com a normalização do argumento fornecido, de forma a assegurar comparações consistentes. Em seguida, o catálogo de reservas é percorrido, sendo identificado o passageiro associado a cada reserva e verificada a sua nacionalidade.

Para reservas válidas, são analisados os voos associados, garantindo que todos existem e não se encontram cancelados. O destino do último voo da reserva é considerado o destino final do passageiro. Os dados são agregados numa tabela de hash que associa cada aeroporto a um conjunto de passageiros distintos, evitando contagens duplicadas.

No final, a tabela é percorrida para identificar o aeroporto com maior número de passageiros. Em caso de empate, é selecionado o aeroporto com o menor código IATA em ordem lexicográfica. Caso não existam passageiros com a nacionalidade indicada, é retornada uma linha vazia.

A complexidade da query é $O(R \cdot m)$, onde R representa o número de reservas e m o número médio de voos por reserva.

2.7 Otimizações e Limitações

Ao longo do desenvolvimento foram aplicadas várias estratégias de encapsulamento, garantindo que os dados internos das estruturas apenas são acedidos através de funções específicas. Esta abordagem contribui para a robustez do sistema e reduz o risco de erros. Foram igualmente aplicadas otimizações ao nível das estruturas de dados e dos algoritmos utilizados, assegurando um bom compromisso entre desempenho e clareza do código.

2.8 Dificuldades Sentidas

As principais dificuldades encontradas durante esta fase estiveram relacionadas com a gestão de memória, nomeadamente na deteção de *memory leaks*, e com a necessidade de adaptar continuamente as estruturas de dados para responder a queries cada vez mais complexas, mantendo simultaneamente um bom nível de encapsulamento. Também foram sentidas dificuladades na execução das queries, principalmente na 4, e na adaptação ao *large dataset*. Sentimos um pouco de dificuldade em gerir o tempo agora na 2º fase.

3 Conclusão

Em conclusão, a 2ª fase do projeto permitiu consolidar o trabalho desenvolvido anteriormente, resultando numa aplicação funcional e bem estruturada. Apesar dos desafios encontrados, foram cumpridos os principais objetivos definidos no enunciado, estabelecendo uma base sólida e consistente para a avaliação final do projeto.

Acreditamos que a boa organização e colaboração dentro do grupo nos permitiram ter um bom desenvolvimento do projeto. Esta experiência contribuiu para o reforço dos nossos conhecimentos em gestão de memória, modularidade, encapsulamento e programação em C.